université
PARIS-SACLAY

EVALUATING hadoop MAPREDUCE FOR PURCHASE DATA PROCESSING

Local VS. Distributed Execution

Author:
Akshay KHOKAN

Supervised by :
Nazim AGOULMINE

M2 GENIOMHE-AI

Table of contents

ABSTRACT..... 2
PROCESSING WITHOUT HADOOP.....2
1. Setup of Mapper and Reducer..... 2
2. Test of Mapper and Reducer on the dataset..... 3
PROCESSING WITH HADOOP.....4
1. Setup of the HDFS environment..... 4
2. Run in Hadoop : Vanilla..... 5
3. Run in Hadoop : Purchases 2.0..... 7
4. Run in Hadoop : Purchases 2.1..... 10
5. Run in Hadoop : Purchases 2.2..... 10
6. Run in Hadoop : Purchases 3.0..... 13
CONCLUSION..... 17
REFERENCES.....18

Evaluating Hadoop MapReduce for Purchase Data Processing: Local VS. Distributed Execution

Akshay KHOKAN [1]

[1] Master's Student (Year 2) in GENIOMHE-AI, Paris-Saclay University

ABSTRACT

The aim of this project was to explore Hadoop [1] and the MapReduce model for processing and analyzing purchase data. The implementation of a mapper and a reducer in Python allowed the calculation of dates, times, cities, items, payment methods and total purchase cost. Moreover, the project involved using HDFS (Hadoop Distributed File System) to store and access the input and output data, while YARN was used to manage and execute the processing jobs. The local execution with the Python implementation was then compared to the Hadoop approach to evaluate their performance. Finally, the use of R was necessary to visualize the results through statistical summaries and plots. The report doesn't contain any information about the setup of Hadoop itself, as the goal here is to evaluate and understand the approach (with its advantages and limitations).

PROCESSING WITHOUT HADOOP

Before using Hadoop to process the purchase data, the goal is to understand how Mapper and Reducer are used to do data processing.

1. Setup of Mapper and Reducer

In order to write the respective scripts for Mapper.py and Reducer.py, the Nano editor will be used. Python 3.14.7 is used here.

```
  GNU nano 7.2                          mapper.py
#!/usr/bin/python
import sys

for line in sys.stdin:
        data = line.strip().split("\t")
        if len(data)==6:
                data, time, store, item, cost, payment = data
                print("{0}\t{1}".format(store,cost))
```

```
 GNU nano 7.2                          reducer.py
#!/usr/bin/python
import sys

salesTotal = 0
oldKey = None

for line in sys.stdin:
        data = line.strip().split("\t")
        if len(data) != 2:
                continue
        thisKey, thisSale = data
        if oldKey and oldKey != thisKey:
                print("{0}\t{1}".format(oldKey, salesTotal))
                salesTotal = 0

        oldKey = thisKey
        salesTotal += float(thisSale)

if oldKey !=None:
        print(oldKey, "\t", salesTotal)
```

2. Test of Mapper and Reducer on the dataset

The following output is obtained after applying Mapper.py on the dataset, sorting the intermediate key-value pairs, and then applying Reducer.py to aggregate the purchase amounts for each city. Only the first 10 lines are shown.

```
head -10 purchases.txt | python mapper.py | sort | python
reducer.py

Austin    379.6
Corpus Christi    25.38
Fort Worth    367.45
New York    296.8
Omaha    235.63
Pittsburgh    493.51
San Diego    66.08
San Jose    214.05
Stockton      247.18
```

Here are interesting statistics obtained with R about the whole dataset, after using Mapper.py and Reducer.py.

```
> data <- read.delim("purchases.txt",header = FALSE,sep = "\t")
> summary(data)
       City         Amount
 Length   :103   Min.   : 9919560
                 Median :10047053
                 Mean   :10043281
                 Max.   :10190080

> dataCity[which.min(dataAmount)]
[1] "Chandler"
> dataCity[which.max(dataAmount)]
[1] "Philadelphia"
```

The 103 cities have a median of 10,047,053$ worth of earning. Chandler has the lowest total purchase amount (9,919,560$) whereas Philadelphia has the highest (10,190,080$). [2]

PROCESSING WITH HADOOP

The following section uses Hadoop to process the dataset using the MapReduce framework. The Mapper and Reducer tasks will be executed within the Hadoop environment to process the data.

1. Setup of the HDFS environment

Before processing the dataset with Hadoop, the HDFS environment must be configured and running. The dataset is transferred from the local machine to the Hadoop container and uploaded to HDFS using the Hadoop commands.

```
docker cp purchases.txt hadoop:/tmp/purchases.txt
Successfully copied 211MB (transferred 211MB) to
hadoop:/tmp/purchases.txt
docker exec -it hadoop bash
root@15031c27bfa3:/# cd $HADOOP_PREFIX
root@15031c27bfa3:/usr/local/hadoop# ./bin/hdfs dfs -mkdir proj2
root@15031c27bfa3:/usr/local/hadoop# ./bin/hdfs dfs -put
/tmp/purchases.txt proj2
```

The file's presence in HDFS is then verified to ensure that the dataset is ready for MapReduce processing.

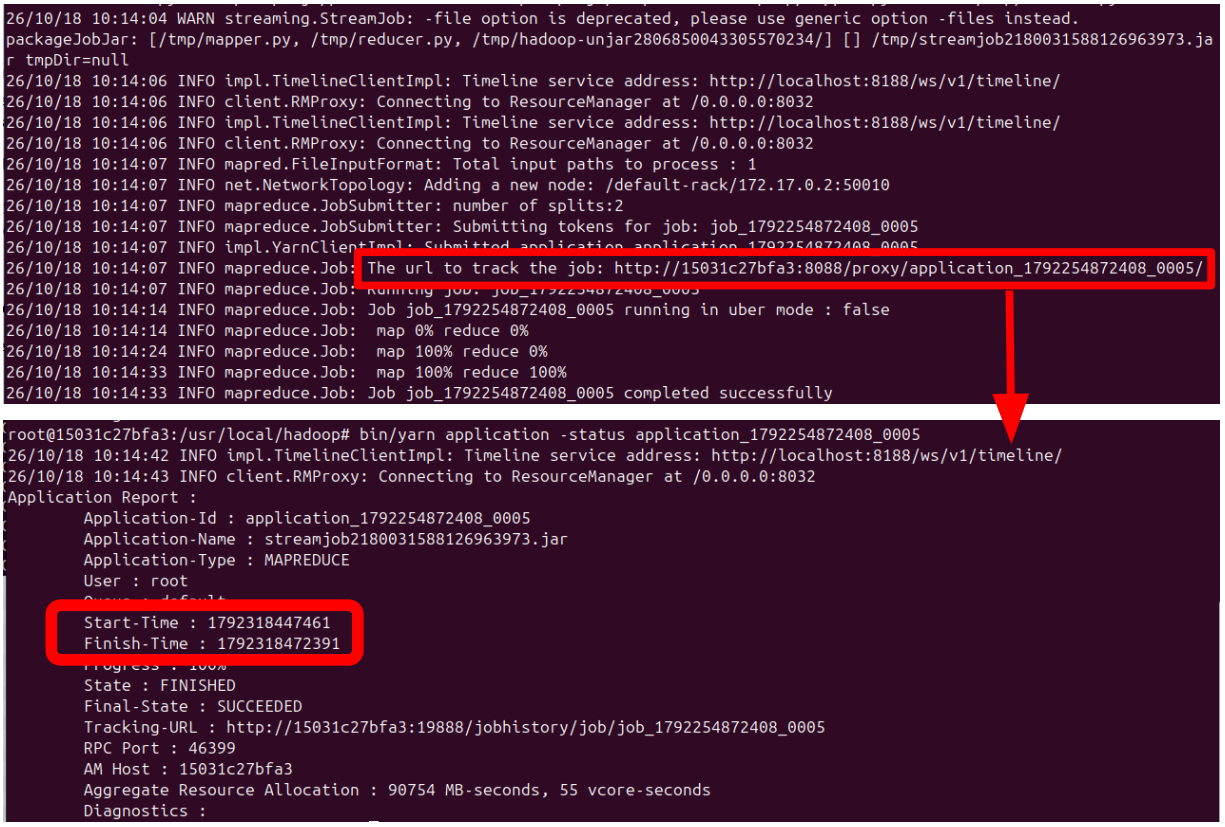
```
root@15031c27bfa3:/usr/local/hadoop# ./bin/hdfs dfs -ls proj2
Found 1 item
-rw-r--r--   1 root supergroup  211312924 2026-10-17 18:33
proj2/purchases.txt
```

2. Run in Hadoop : Vanilla

The dataset can be processed using HDFS, thanks to the configuration completed before, with the following command line, to **<u>calculate the total income per city.</u>**

```
root@15031c27bfa3:/usr/local/hadoop# ./bin/hadoop jar
share/hadoop/tools/lib/hadoop-streaming-2.7.2.jar   -mapper
mapper.py   -reducer reducer.py   -input proj2/purchases.txt
-output proj2/output   -file /tmp/mapper.py   -file /tmp/reducer.py
```

During the execution of the job, an URL to track it was given. Current state of the job, start and finish times (in milliseconds) are obtained below.



Given the start and finish times obtained before, the estimated time of the job in seconds is :

$$\frac{1792318472391 - 1792318447461}{1000} = \frac{24930}{1000}$$
$$= \frac{2493}{100}$$
$$= 24.93 \text{ s}$$

The statistics are another point to check.

```
> summary(data)
        City         Amount
 Length   :103   Min.   : 9919560
 N.unique :103   1st Qu.:10006300
 N.blank  :  0   Median :10048681
 Min.nchar:  4   Mean   :10043274
 Max.nchar: 49   3rd Qu.:10079736
                 Max.   :10190080
                 NAs    :1
>
> dataCity[which.min(dataAmount)]
[1] "Chandler"
> dataCity[which.max(dataAmount)]
[1] "Philadelphia"
```

The 103 cities have a median of 10,048,681$ worth of earning. Chandler has the lowest total purchase amount (9,919,560$) whereas Philadelphia has the highest (10,190,080$).

The data processing with HDFS gave similar results to the one made locally. Indeed, the **median slightly changed** (increased with HDFS, it might be because of data partitioning) and **the execution time using HDFS is almost the same (25,176 VS. 24,93 seconds)** as the local data processing.

```
time bash -c 'cat purchases.txt | python mapper.py | sort | python
reducer.py'

real    0m23,340s
user    0m25,176s
sys     0m1,022s
```

Finally, the following code block below shows the first ten lines obtained after the local (blue) and HDFS (green) data processing. The values are the same (just rounded).

```
root@15031c27bfa3:/usr/local/hadoop# ./bin/hdfs dfs -cat
proj2/output/part-00000 | head -n 10

cat -10 local_output.txt | sort | head -n 10


Albuquerque    10052311.42   10052311.420000007
Anaheim    10076416.36    10076416.35999996
Anchorage    9933500.4    9933500.399999978
Arlington    10072207.97    10072207.969999986
Atlanta    9997146.7        9997146.699999938
Aurora    9992970.92        9992970.920000073
Austin    10057158.9        10057158.899999972
Bakersfield    10031208.92    10031208.919999935
Baltimore    10096521.45      10096521.449999953
Baton Rouge    10131273.23    10131273.230000008
```

3. Run in Hadoop : Purchases 2.0

The previous Mapper file is duplicated then modified, with nano to **calculate the totals per product category.** The Reducer file remains untouched, as the logic stays the same.

```
root@15031c27bfa3:/usr/local/hadoop# cp /tmp/mapper.py /tmp/mapper1.py
```

```
 GNU nano 2.2.6                File: /tmp/mapper1.py

#!/usr/bin/python
import sys

for line in sys.stdin:
        data = line.strip().split("\t")
        if len(data)==6:
                data, time, store, item, cost, payment = data
                print("{0}\t{1}".format(item,cost))
```

```
time bash -c 'cat /tmp/purchases.txt | python /tmp/mapper1.py |
sort | python /tmp/reducer.py'

Baby    57491808.44
Books    57450757.91
CDs    57410753.04
Cameras    57299046.64
Children's Clothing    57624820.94
Computers    57315406.32
Consumer Electronics    57452374.13
Crafts    57418154.5
DVDs    57649212.14
Garden    57539833.11
Health and Beauty    57481589.56
Men's Clothing    57621279.04
Music    57495489.7
Pet Supplies    57197250.24
Sporting Goods    57599085.89
Toys    57463477.11
Video Games    57513165.58
("Women's Clothing", '\t', 57434448.96999975)

real    0m11.752s
user    0m12.034s
sys    0m1.329s
```

Since the time command was used to measure execution locally, the URL provided by Hadoop will not be used to estimate the execution time further.

```
root@15031c27bfa3:/usr/local/hadoop# time bash -c './bin/hadoop
jar share/hadoop/tools/lib/hadoop-streaming-2.7.2.jar   -mapper
mapper1.py   -reducer reducer.py   -input proj2/purchases.txt
-output proj2/output   -file /tmp/mapper1.py   -file
/tmp/reducer.py'

...

real    0m29.354s
user    0m11.318s
sys    0m1.059s
```

```
root@15031c27bfa3:/usr/local/hadoop# ./bin/hdfs dfs -cat
proj2/output/part-00000

Baby    57491808.44
Books    57450757.91
CDs    57410753.04
Cameras    57299046.64
Children's Clothing    57624820.94
Computers    57315406.32
Consumer Electronics    57452374.13
Crafts    57418154.5
DVDs    57649212.14
Garden    57539833.11
Health and Beauty    57481589.56
Men's Clothing    57621279.04
Music    57495489.7
Pet Supplies    57197250.24
Sporting Goods    57599085.89
Toys    57463477.11
Video Games    57513165.58
("Women's Clothing", '\t', 57434448.969999254)
```

The outputs are the same with or without HDFS, however using **HDFS here is slightly faster (~1 second difference)**.

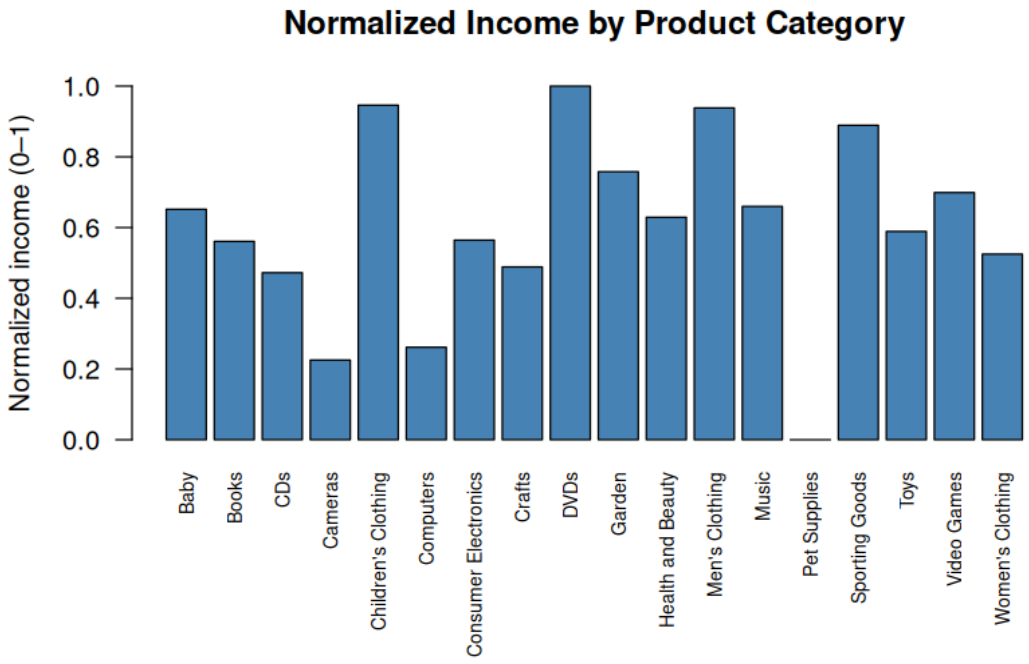


Figure 1. Normalized income by Product Category [3]

The min normalized value is 0 for Pet Supplies, and the maximum is 1 for DVDs. The other values belong to [min, max], to see the distribution of the sales for each category.

4. Run in Hadoop : Purchases 2.1

This section covers the **calculation of the total income per city (store)**, which was already done in section 2. Run in Hadoop : Vanilla.

5. Run in Hadoop : Purchases 2.2

This section covers the **calculation of the most expensive payment per city (store)**. The initial Mapper file is kept, but the Reducer file is duplicated and then modified with Nano.

```
 GNU nano 2.2.6                File: /tmp/reducer1.py

#!/usr/bin/python
import sys

oldKey = None
maxPayment = 0.0

for line in sys.stdin:
    data = line.strip().split("\t")
    if len(data) != 2:
        continue

    city, payment = data

    try:
        payment = float(payment)
    except ValueError:
        continue

    if oldKey is not None and oldKey != city:
        print("{0}\t{1}".format(oldKey, maxPayment))
        maxPayment = 0.0

    if oldKey != city:
        oldKey = city

    maxPayment = max(maxPayment, payment)

if oldKey is not None:
    print("{0}\t{1}".format(oldKey, maxPayment))
```

```
root@15031c27bfa3:/usr/local/hadoop# time bash -c './bin/hadoop
jar share/hadoop/tools/lib/hadoop-streaming-2.7.2.jar   -mapper
mapper.py   -reducer reducer1.py   -input proj2/purchases.txt
-output proj2/output   -file /tmp/mapper.py   -file
/tmp/reducer.py'
```

```
real    0m31.928s
user    0m11.409s
sys    0m1.128s
```

Above is the execution time of the HDFS job and below is the one done locally, without HDFS.

```
root@15031c27bfa3:/usr/local/hadoop# time bash -c 'cat
/tmp/purchases.txt | python /tmp/mapper.py | sort | python
/tmp/reducer1.py'

...

real    0m12.333s
user    0m12.680s
sys    0m1.212s
```

The outputs are the same with or without HDFS, however using **HDFS here is slightly faster (~1 second difference)**. As the outputs are quite concerning (~500$ everywhere), a big part of the Hadoop output is shown.

```
root@15031c27bfa3:/usr/local/hadoop# ./bin/hdfs dfs -cat
proj2/output2/part-00000
Albuquerque    499.98
Anaheim    499.98
Anchorage    499.99
Arlington    499.95
Atlanta    499.96
Aurora    499.97
Austin    499.97
Bakersfield    499.97
Baltimore    499.99
Baton Rouge    499.98
Birmingham    499.99
Boise    499.98
Boston    499.99
Buffalo    499.99
Chandler    499.98
Charlotte    499.98
```

```
...

Sacramento    499.96
Saint Paul    499.98
San Antonio    499.98
San Bernardino    499.98
San Diego    499.98
San Francisco    499.97
San Jose    499.99
Santa Ana    499.97
Scottsdale    499.98
Seattle    499.99
Spokane    499.99
St. Louis    499.99
St. Petersburg    499.95
Stockton    499.98
Tampa    499.99
Toledo    499.98
Tucson    499.98
Tulsa    499.96
Virginia Beach    499.98
Washington    499.98
Wichita    499.97
Winston-Salem    499.98
```

The most expensive payment per city is 500$ on average (Obtained with R).

```
> max_cost_by_city <- read.delim(
+  "purchases3.txt",
+  header = FALSE,
+  sep = "\t",
+  stringsAsFactors = FALSE
+ )

> summary(max_cost_by_city)
        city        max_cost
 Length   :103   Min.   :499.9
 N.unique :103   1st Qu.:500.0
 N.blank  :  0   Median :500.0
 Min.nchar:  4   Mean   :500.0
 Max.nchar: 16   3rd Qu.:500.0
                 Max.   :500.0
```

6. Run in Hadoop : Purchases 3.0

This section uses YARN to process and summarize every purchase data with MapReduce. That is, the mapper extracts the six fields (date, time, city, item, cost and payment method). The reducer then groups the same values and calculates their total number of occurrences, except for the cost. Indeed, the overall cost is calculated once.

```
 GNU nano 2.2.6                File: /tmp/mapper2.py

#!/usr/bin/python
import sys

for line in sys.stdin:
    data = line.strip().split("\t")

    if len(data) != 6:
        continue

    date, time, city, item, cost, payment = data

    fields = [
        ("date", date),
        ("time", time),
        ("city", city),
        ("item", item),
        ("payment", payment)
    ]

    for field, value in fields:
        print("{0}:{1}\t1".format(field, value))

    try:
        cost = float(cost)
        print("cost:total\t{0}".format(cost))
    except ValueError:
        continue
```

The mapper file was changed here with Nano, to extract the six fields.

```
  GNU nano 2.2.6                File: /tmp/reducer2.py

#!/usr/bin/python
import sys

old_key = None
total = 0.0

for line in sys.stdin:
    data = line.strip().split("\t")

    if len(data) != 2:
        continue

    key, value = data

    try:
        value = float(value)
    except ValueError:
        continue

    if old_key is not None and key != old_key:
        print("{0}\t{1:g}".format(old_key, total))
        total = 0.0

    old_key = key
    total += value

if old_key is not None:
    print("{0}\t{1:g}".format(old_key, total))
```

The reducer file was changed here to group identical keys and compute their total number of occurrences, with the total cost.

```
root@15031c27bfa3:/usr/local/hadoop# time bash -c 'cat
/tmp/purchases.txt | python /tmp/mapper2.py | sort | python
/tmp/reducer2.py'

...

real    0m33.286s
user    0m33.966s
sys     0m2.668s
```

The execution **in local took 33 seconds (above),** whereas **with YARN it took only 7.682 seconds (below).**

```
root@15031c27bfa3:/usr/local/hadoop# time bash -c './bin/yarn jar
/usr/local/hadoop/share/hadoop/tools/lib/hadoop-streaming-2.7.0.ja
r \
>   -mapper mapper2.py \
>   -reducer reducer2.py \
>   -input proj2/purchases.txt \
>   -output proj2/output_everything \
>  -file /tmp/mapper2.py  \
>  -file /tmp/reducer2.py '

real    0m53.916s
user    0m7.682s
sys    0m0.713s
```

The two approaches give the same outputs, with 1032 lines each :

- 18 items (Baby to Women's Clothing),
- 5 payment methods (Amex, Cash, Discover, MasterCard, Visa) ,
- 366 daily dates (2012-01-01 to 2012-12-31),
- 103 cities
- 540 timestamps (09:00 to 17:59),
- One total cost

As 1032 lines are quite complicated to visualize, the output was cut into different sections, in order to have proper data visualization.

```
// hello.txt is the output file
root@15031c27bfa3:/usr/local/hadoop# grep time /tmp/hello.txt >
/tmp/time.txt
root@15031c27bfa3:/usr/local/hadoop# grep item /tmp/hello.txt >
/tmp/item.txt
root@15031c27bfa3:/usr/local/hadoop# grep date /tmp/hello.txt >
/tmp/date.txt
root@15031c27bfa3:/usr/local/hadoop# grep payment /tmp/hello.txt >
/tmp/payment.txt
root@15031c27bfa3:/usr/local/hadoop# grep city /tmp/hello.txt >
/tmp/city.txt
```

Filtering the output file by keys to create new files then copying these files locally were necessary to visualize the data.

```
docker cp hadoop:/tmp/time.txt ~/time.txt
docker cp hadoop:/tmp/item.txt ~/item.txt
docker cp hadoop:/tmp/date.txt ~/date.txt
docker cp hadoop:/tmp/payment.txt ~/payment.txt
docker cp hadoop:/tmp/city.txt ~/city.txt
```

For the date distribution, the beginning of January 2012 has the highest daily average of purchases, whereas the beginning of December 2012 has the lowest one.

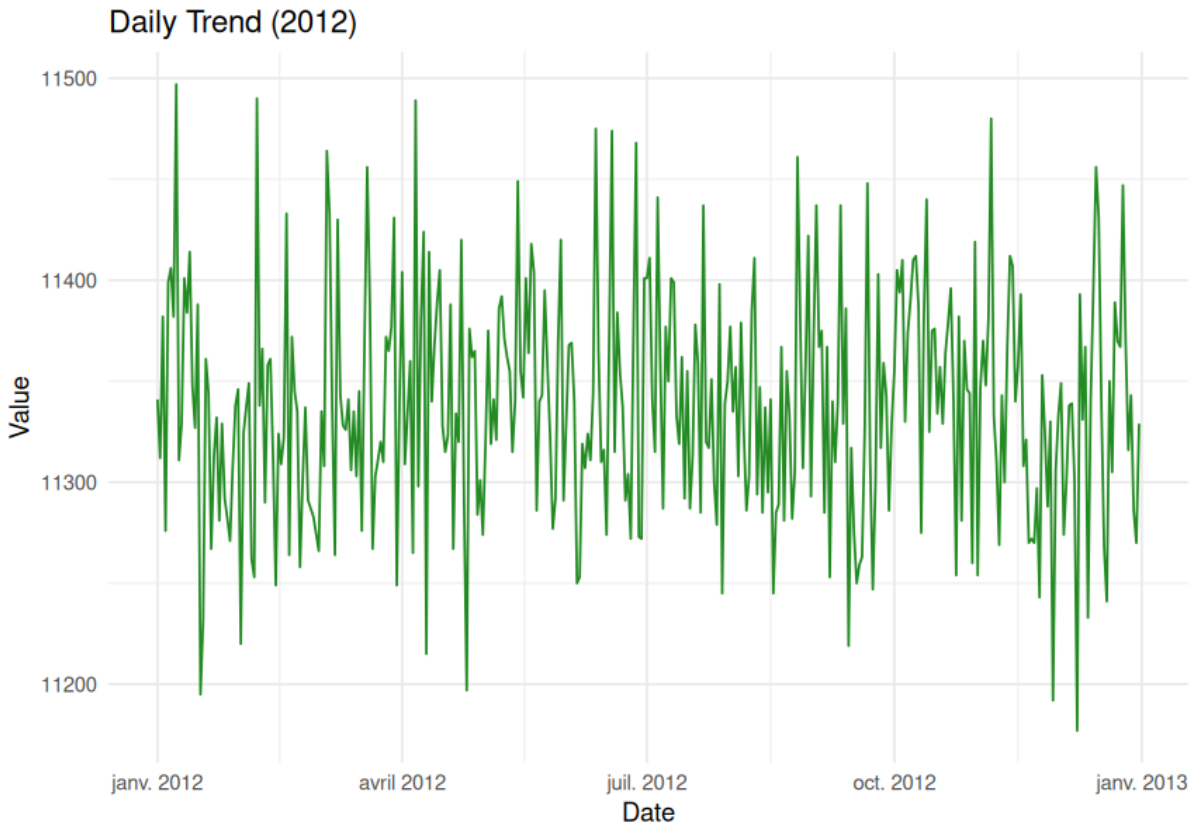


Figure 2. Daily trend of Purchases [3]

The following R code block gives information about the purchases based on the cities, the payment methods and the time.

```
> city_data[which.min(city_data$value), ]
                key value        city
46 city:Jersey City 39612 Jersey City
> city_data[which.max(city_data$value), ]
                 key value         city
72 city:Philadelphia 40748 Philadelphia
```

```
> payment_data[which.min(payment_data$value), ]
           key  value payment
1 payment:Amex 826535    Amex
> payment_data[which.max(payment_data$value), ]
           key  value payment
2 payment:Cash 828770    Cash

>     heures     <-     as.numeric(sub("^time:([0-9]{2}):.*",     "\\1",
time_data$key))
> sum(time_data$value[heures < 12])
[1] 1378890
> sum(time_data$value[heures > 12])
[1] 2299680

# This is a snippet of the R script. Actually I used regex twice, and I
used the R guide to do it. [4]
```

The total purchase cost reached 1.034 billion $. Among the cities, **Jersey City recorded the lowest number of purchases** (39,162), while **Philadelphia reached the highest one** (40,748). For payment methods, **American Express registered the lowest use rate** (826,535), whereas **Cash has the highest one** (828,770). People are more likely to **buy in the afternoon** (2,299,680) **than in the morning** (1,378,890).

CONCLUSION

The project demonstrated how Hadoop and the MapReduce model can be used to process and analyze purchase data. An important part of this project was comparing the performance of Hadoop with local execution. Results showed that Hadoop introduces additional overhead due to job initialization and distributed processing. For small datasets and small queries, the difference of speed is neglectable, however Hadoop is made to handle much larger datasets. The YARN execution was a perfect showcase of the benefits of using Hadoop rather than a normal local execution.

REFERENCES

1. **Apache Software Foundation**. *Hadoop Streaming, Hadoop 2.7.0 Documentation.* https://hadoop.apache.org/docs/r2.7.0/hadoop-streaming/HadoopStreaming.html
2. **Sanderson, S. P.**. *How to Select Row with Max Value in Specific Column in R, Steve's Data Tips and Tricks.* https://www.spsanderson.com/steveondata/posts/2025-10-06/
3. **R Core Team**. *Bar Plots, R Documentation.* https://search.r-project.org/R/refmans/graphics/html/barplot.html
4. **R Core Team**. *Pattern Matching and Replacement, R Documentation.* https://stat.ethz.ch/R-manual/R-devel/library/base/html/grep.html